

例题 10-5 GCD 等于 XOR (GCD XOR, ACM/ICPC Dhaka 2013, UVa12716)

输入整数 $n (1 \leq n \leq 30000000)$, 有多少对整数 (a, b) 满足: $1 \leq b \leq a \leq n$, 且 $\gcd(a, b) = a \text{ XOR } b$ 。例如 $n=7$ 时, 有 4 对: $(3, 2), (5, 4), (6, 4), (7, 6)$ 。

【分析】

本题看上去很难找到简洁的数学公式, 因为 $\gcd$ 和 xor 看上去似乎毫不相干。不过 xor 的好处是: $a \text{ xor } b = c$, 则 $a \text{ xor } c = b$, 所以可以枚举 a 和 c , 然后算出 $b = a \text{ xor } c$, 最后验证一下是否有 $\gcd(a, b) = c$ 。时间复杂度如何? 因为 c 是 a 的约数, 所以和素数筛法类似, 时间复杂度为 $n/1 + n/2 + \dots + n/n = O(n \log n)$ 。再加上 $\gcd$ 的时间复杂度为 $O(\log n)$, 所以总的时间复杂度为 $O(n(\log n)^2)$ 。

我们还可以做得更好。上述程序写出来之后, 可以打印一些满足 $\gcd(a, b) = a \text{ xor } b = c$ 的三元组 (a, b, c) , 然后很容易发现一个现象: $c = a - b$ 。

证明如下: 不难发现 $a - b \leq a \text{ xor } b$, 且 $a - b \geq c$ 。假设存在 c 使得 $a - b > c$, 则 $c < a - b \leq a \text{ xor } b$, 与 $c = a \text{ xor } b$ 矛盾。

有了这个结论, 还是沿用上述算法, 枚举 a 和 c , 计算 $b = a - c$, 则 $\gcd(a, b) = \gcd(a, a - c) = c$, 因此只需验证是否有 $c = a \text{ xor } b$, 时间复杂度降为了 $O(n \log n)$ 。

10.2 计数与概率基础

排列与组合是最基本的计数技巧。本节介绍一些基本的相关知识和方法, 供读者参考。

加法原理。做一件事情有 n 个办法, 第 i 个办法有 p_i 种方案, 则一共有 $p_1 + p_2 + \dots + p_n$ 种方案。

乘法原理。做一件事情有 n 个步骤, 第 i 个步骤有 p_i 种方案, 则一共有 $p_1 p_2 \dots p_n$ 种方案。

乘法原理是加法原理的特殊情况 (按照第一步骤进行分类), 二者都可用于递推。注意应用加法原理的关键是分类: 各类别之间必须没有重复、没有遗漏。如果有重复, 可以使用容斥原理。

容斥原理。假设班里有 10 个学生喜欢数学, 15 个学生喜欢语文, 21 个学生喜欢编程, 一共有多少个学生呢? 是 $10 + 15 + 21 = 46$ 个吗? 不是的, 因为有些学生可能同时喜欢数学和语文, 或者语文和编程, 甚至还可能有三者都喜欢的。为了叙述方便, 将喜欢语文、数学、编程的学生集合分别用 A, B, C 表示, 则学生总数等于 $|A \cup B \cup C|$ 。刚才已经说了, 如果把这 3 个集合的元素个数 $|A|, |B|, |C|$ 直接加起来, 会有一些元素重复统计了, 因此需要扣掉 $|A \cap B|, |B \cap C|, |C \cap A|$, 但这样一来, 又有一小部分多扣了, 需要加回来: $|A \cap B \cap C|$ 。这样, 就得到了一个公式:

$$|A \cup B \cup C| = |A| + |B| + |C| - |A \cap B| - |B \cap C| - |C \cap A| + |A \cap B \cap C|$$

一般地, 对于任意多个集合, 都可以列出这样一个等式, 其中左边是所有集合的并的元素个数, 右边是这些集合的“各种搭配”。每个“搭配”都是若干个集合的交集, 且每一项前面的正负号取决于集合的个数——奇数个集合为正, 偶数个集合为负。

有重复元素的全排列。有 k 个元素，其中第 i 个元素有 n_i 个，求全排列个数。

【分析】

令所有 n_i 之和为 n ，再设答案为 x 。首先做全排列，然后把所有元素编号，其中第 s 种元素编号为 $1 \sim n_s$ （例如，有 3 个 a ，两个 b ，先排列成 $aabba$ ，然后可以编号为 $a_1a_3b_2b_1a_2$ ）。这样做以后，由于编号后所有元素均不相同，方案总数为 n 的全排列数 $n!$ 。根据乘法原理，得到了一个方程： $n_1!n_2!n_3!\cdots n_k!x=n!$ ，移项即可。

可重复选择的组合。有 n 个不同元素，每个元素可以选多次，一共选 k 个元素，有多少种方法？例如， $n=3, k=2$ 时有 6 种： $(1,1), (1,2), (1,3), (2,2), (2,3), (3,3)$ 。

【分析】

设第 i 个元素选 x_i 个，问题转化为求方程 $x_1+x_2+\cdots+x_n=k$ 的非负整数解的个数。令 $y_i=x_i+1$ ，则答案为 $y_1+y_2+\cdots+y_n=k+n$ 的正整数解的个数。想象有 $k+1$ 个数字“1”排成一排，则问题等价于：把这些“1”分成 n 个部分，有多少种方法？这相当于在 $k+n-1$ 个“候选分隔线”中选 $n-1$ 个，即 $C(k+n-1, n-1)=C(n+k-1, k)$ 。

10.2.1 杨辉三角与二项式定理

组合数 C_n^m 在组合数学中占有重要地位。与组合数相关的最重要的两个内容是杨辉三角和二项式定理。如图 10-1 所示就是一个杨辉三角。

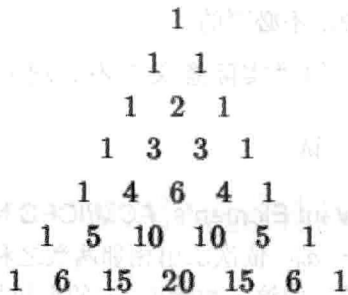


图 10-1 杨辉三角

另一方面，把 $(a+b)^n$ 展开，将得到一个关于 x 的多项式：

$$(a+b)^0 = 1$$

$$(a+b)^1 = a+b$$

$$(a+b)^2 = a^2 + 2ab + b^2$$

$$(a+b)^3 = a^3 + 3a^2b + 3ab^2 + b^3$$

$$(a+b)^4 = a^4 + 4a^3b + 6a^2b^2 + 4ab^3 + b^4$$

系数正好和杨辉三角一致。一般地，有二项式定理：

$$(a+b)^n = \sum_{k=0}^n C_n^k a^{n-k} b^k$$

这不难理解： $(a+b)^n$ 是 n 个括号连乘，每个括号里任选一项乘起来都会对最后的结果

有一个贡献。如果选了 k 个 a , 就一定会选 $n-k$ 个 b , 最后的项自然就是 $a^{n-k}b^k$ 。而从 n 个 a 里选 k 个 (同时也相当于 n 个 b 里选 $n-k$ 个) 有 C_n^k 种方法, 这也是组合数的定义。

给定 n , 如何求出 $(a+b)^n$ 中所有项的系数呢? 一个方法是用递推, 根据杨辉三角中不难发现的规律, 可以写出如下程序:

```
memset(C, 0, sizeof(C));
for(int i = 0; i <= n; i++) {
    C[i][0] = 1;
    for(int j = 1; j <= i; j++) C[i][j] = C[i-1][j-1] + C[i-1][j];
}
```

但遗憾的是, 这个算法的时间复杂度是 $O(n^2)$ ——尽管只用了杨辉三角的第 n 行的 $n+1$ 个元素, 却把全部 n 行的 $O(n^2)$ 个元素都计算了一遍。

另一个方法是利用等式 $C_n^k = \frac{n-k+1}{k} C_n^{k-1}$, 从 $C_n^0 = 1$ 开始从左到右递推, 例如:

```
C[0] = 1;
for(int i = 1; i <= n; i++) C[i] = C[i-1]*(n-i+1)/i;
```

注意, 应该先乘后除, 因为 $C[i-1]/i$ 可能不是整数。但这样一来增加了溢出的可能性——即使最后结果在 int 或 long long 范围之内, 乘法也可能溢出。如果担心这样的情况出现, 可以先约分, 不过一般来说是不必要的。

尽管等式 $C_n^k = \frac{n-k+1}{k} C_n^{k-1}$ 的“实际意义”不是很明显, 却很容易用组合数公式

$C_n^k = \frac{n!}{k!(n-k)!}$ 证明, 读者不妨一试。

例题 10-6 无关的元素 (Irrelevant Elements, ACM/ICPC NEERC 2004, UVa1635)

对于给定的 n 个数 $a_1, a_2, \dots, a_n$, 依次求出相邻两数之和, 将得到一个新数列。重复上述操作, 最后结果将变成一个数。问这个数除以 m 的余数与哪些数无关? 例如 $n=3, m=2$ 时, 第一次求和得到 a_1+a_2, a_2+a_3 , 再求和得到 $a_1+2a_2+a_3$, 它除以 2 的余数和 a_2 无关。 $1 \leq n \leq 10^5, 2 \leq m \leq 10^9$ 。

【分析】

显然最后的求和式是 $a_1, a_2, \dots, a_n$ 的线性组合。设 a_i 的系数为 $f(i)$, 则和式除以 m 的余数与 a_i 无关, 当且仅当 $f(i)$ 是 i 的倍数。不妨看一个简单的例子:

$$\begin{array}{ccccccccc}
 & a_1 & & a_2 & & a_3 & & a_4 & & a_5 \\
 & a_1 + a_2 & & a_2 + a_3 & & a_3 + a_4 & & a_4 + a_5 & & \\
 & a_1 + 2a_2 + a_3 & & a_2 + 2a_3 + a_4 & & a_3 + 2a_4 + a_5 & & & & \\
 & a_1 + 3a_2 + 3a_3 + a_4 & & a_2 + 3a_3 + 3a_4 + a_5 & & & & & & \\
 & a_1 + 4a_2 + 6a_3 + 4a_4 + a_5 & & & & & & & &
 \end{array}$$

看到最后的结果, 你想到了什么? 没错, “1 4 6 4 1”正是杨辉三角的第 5 行! 不难证明, 在一般情况下, 最后 a_i 的系数是 C_{n-1}^{i-1} 。这样, 问题就变成了 $C_{n-1}^0, C_{n-1}^1, \dots, C_{n-1}^{n-1}$ 中有

哪些是 m 的倍数。

还记得二项式展开的方法吗? 理论上, 利用此方法可以递推出所有 C_{n-1}^{i-1} , 但它们太大了, 必须用高精度才能存得下。但此问题中所关心的只是“哪些是 m 的倍数”, 受到数论部分中的启发, 只需要依次计算 m 的唯一分解式中各个素因子在 C_{n-1}^{i-1} 中的指数即可完成判断。这些指数仍然可以用 $C_n^k = \frac{n-k+1}{k} C_n^{k-1}$ 递推, 并且不会涉及高精度。有的读者可能会尝试直接递推每个系数除以 m 的余数, 但遗憾的是, 递推式中有除法, 而模 m 意义下的逆并不一定存在。

10.2.2 数论中的计数问题

约数的个数。 给出正整数 n 的唯一分解式 $n = p_1^{a_1} p_2^{a_2} p_3^{a_3} \cdots p_k^{a_k}$, 求 n 的正约数的个数。

【分析】

不难看出, n 的任意正约数也只能包含 p_1, p_2, p_3 等素因子, 而不能有新的素因子出现。对于 n 的某个素因子 p_i , 它在所求约数中的指数可以是 $0, 1, 2, \dots, a_i$ 共 a_i+1 种情况, 而且不同的素因子之间相互独立。根据乘法原理, n 的正约数个数为:

$$\prod_{i=1}^k (a_i + 1) = (a_1 + 1)(a_2 + 1) \cdots (a_k + 1)$$

小于 n 且与 n 互素的整数个数。 给出正整数 n 的唯一分解式 $n = p_1^{a_1} p_2^{a_2} p_3^{a_3} \cdots p_k^{a_k}$, 求 $1, 2, 3, \dots, n$ 中与 n 互素的数的个数。

【分析】

用容斥原理。首先从总数 n 中分别减去是 $p_1, p_2, \dots, p_k$ 的倍数的个数 (对于素数 p 来说, “与 p 互素” 和 “不是 p 的倍数” 等价), 即 $n - \frac{n}{p_1} - \frac{n}{p_2} - \cdots - \frac{n}{p_k}$, 然后加上 “同时是两个素因子的倍数” 的个数 $\frac{n}{p_1 p_2} + \frac{n}{p_1 p_3} + \cdots + \frac{n}{p_{k-1} p_k}$, 再减去 “同时是 3 个素因子的倍数” —— 写成一个 “学术味比较浓” 的公式就是:

$$\varphi(n) = \sum_{S \subseteq \{p_1, p_2, \dots, p_k\}} (-1)^{|S|} \prod_{p_i \in S} \frac{n}{p_i}$$

这里引入的新记号 $\varphi(n)$ 就是题目中所求的结果, 称为欧拉函数。强烈建议初学者花一些时间理解这个公式。对于 $\{p_1, p_2, \dots, p_k\}$ 的任意子集 S , “不与其中任何一个互素” 的元素个数是 $\frac{n}{\prod_{p_i \in S} p_i}$ 。不过这一项的前面是加号还是减号呢? 这取决于 S 中的元素个数——奇数个就是 “减号”, 偶数个就是 “加号”。

公式已得出, 可计算起来很不方便。如果直接根据公式, 需要计算多达 2^k 项的代数和, 甚至可能比 “暴力枚举 (依次判断 $1 \sim n$ 中每个数是否与 n 互素)” 还要慢。

下一步并不显然。上述公式可以变形成如下的形式:

$$\varphi(n) = n \left(1 - \frac{1}{p_1}\right) \left(1 - \frac{1}{p_2}\right) \cdots \left(1 - \frac{1}{p_k}\right)$$

从而只需要 $O(k)$ 的计算时间, 在刚才的基础上大大提高了效率。为什么这个式子和上一个等价呢? 直接考虑新公式的“展开方式”即可。展开式的每一项是从每个括号各选一个 (选 1 或者 $-\frac{1}{p_i}$), 全部乘起来以后再乘以 n 得到。这不正是最初的推导过程吗?

如果没有给出唯一分解式, 需要用试除法依次判断 $\sqrt{n}$ 内的所有素数是否是 n 的因子。这样, 则需要先生成 $\sqrt{n}$ 内的素数表。但其实并不用这么麻烦: 只需要每次找到一个素因子之后把它“除干净”, 即可保证找到的因子都是素数 (想一想, 为什么)。

```
int euler_phi(int n) {
    int m = (int)sqrt(n+0.5);
    int ans = n;
    for(int i = 2; i <= m; i++) if(n % i == 0) {
        ans = ans / i * (i-1);
        while(n % i == 0) n /= i;
    }
    if(n > 1) ans = ans / n * (n-1);
}
```

1~ n 中所有数的欧拉 ϕ 函数值。并不需要依次计算。可以用与筛法求素数非常类似的方法, 在 $O(n \log \log n)$ 时间内计算完毕, 例如 (原理请读者体会):

```
void phi_table(int n, int* phi) {
    for(int i = 2; i <= n; i++) phi[i] = 0;
    phi[1] = 1;
    for(int i = 2; i <= n; i++) if(!phi[i])
        for(int j = i; j <= n; j += i) {
            if(!phi[j]) phi[j] = j;
            phi[j] = phi[j] / i * (i-1);
        }
}
```

例题 10-7 交表 (Send a Table, UVa10820)

有一道比赛题目, 输入两个整数 x, y ($1 \leq x, y \leq n$), 输出某个函数 $f(x, y)$ 。有位选手想交表 (即事先计算出所有的 $f(x, y)$, 写在源代码里), 但是表太大了, 源代码超过了比赛的限制, 需要精简。

好在那道题目有一个性质, 使得很容易根据 $f(x, y)$ 算出 $f(x*k, y*k)$ (其中 k 是任意正整数), 这样有一些 $f(x, y)$ 就不需要存在表里了。

输入 n ($n \leq 50000$), 你的任务是统计最简的表里有多少个元素。例如, $n=2$ 时有 3 个: $(1, 1), (1, 2), (2, 1)$ 。

【分析】

本题的本质是: 输入 n , 有多少个二元组 (x, y) 满足: $1 \leq x, y \leq n$, 且 x 和 y 互素。不难发现除了 $(1, 1)$ 之外, 其他二元组 (x, y) 中的 x 和 y 都不相等。设满足 $x < y$ 的二元组有 $f(n)$ 个, 那

么答案就是 $2f(n)+1$ 。

对照欧拉函数的定义,可以得到 $f(n)=\phi(2)+\phi(3)+\cdots+\phi(n)$, 时间复杂度为 $O(n\log\log n)$ 。

10.2.3 编码与解码

两个 a 、一个 b 和一个 c 组成的所有串可以按照字典序编号为:

$aabc(1)$ 、 $aacb(2)$ 、 $abac(3)$ 、 $\cdots$ 、 $cbaa(12)$

任给一个字符串, 能否方便地求出它的编号呢? 例如, 输入 $acab$, 则应输出 5。

下面直接求解一般情况的问题(并不限定字母的种类和个数)。设输入串为 S , 记 $d(S)$ 为 S 的各个排列中, 字典序比 S 小的串的个数, 则可以用递推法求解 $d(S)$, 如图 10-2 所示。

其中边上的字母表示“下一个字母”, $f(x)$ 表示多重集 x 的全排列个数。例如, 根据第一个字母, 可以把字典序小于 $caba$ 的字符串分为 3 种: 以 a 开头的, 以 b 开头的, 以 c 开头的, 分别对应 $d(caba)$ 的 3 棵子树。以 a 开头的所有串的字典序都小于 $caba$, 所以剩下的字符可以任意排列, 个数为 $f(cba)$; 同理, 以 b 开头的所有串的字典序也都小于 $caba$, 个数为 $f(caa)$; 以 c 开头的串字典序不一定小于 $caba$, 关键要看后 3 个字符, 因此这部分的个数为 $d(aba)$, 还需要继续往下分。

至于 f 函数的求解, 大部分组合数学书籍中均有介绍: 设字符一共有 k 类, 个数分别为 $n_1, n_2, \cdots, n_k$, 则这个多重集的全排列个数为 $\frac{(n_1 + n_2 + \cdots + n_k)!}{n_1! n_2! \cdots n_k!}$ 。

不难算出, $f(caa) = \frac{(1+2)!}{1!2!} = \frac{6}{2} = 3$, 其他 f 值分别为 $f(cba)=6, f(b)=1$, 故 $d(caba)=f(cba)+f(caa)+f(b)=3+6+1=10$ 。既然“比它小”的个数是 10, 序号自然就是 11 了。

“给物体一个编号”称为编码, 同理也有“解码”, 即根据序号构造出这个物体。这个过程和刚才的很接近: 依次确定各个位置上的字母即可。例如, 要求出序号为 8 (因此有 7 个比它小) 的字符串, 推理过程如图 10-3 所示。

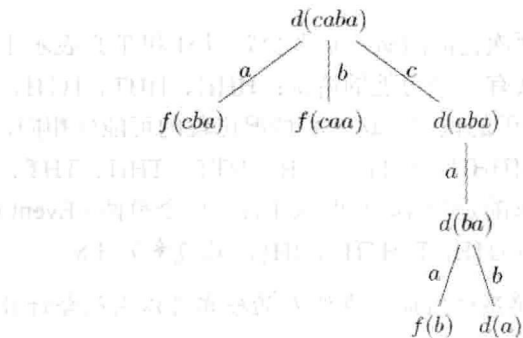


图 10-2 字符串编码的递推过程

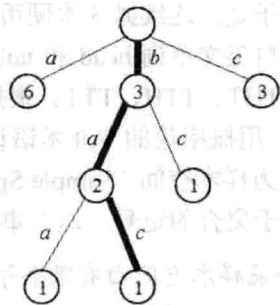


图 10-3 字符串解码的递推过程

例题 10-8 密码 (Password, ACM/ICPC Daejeon 2010, UVa1262)

给两个 6 行 5 列的字母矩阵, 找出满足如下条件的“密码”: 密码中的每个字母在两个矩阵的对应列中均出现。例如, 左数第 2 个字母必须在两个矩阵中的左数第 2 列中均出现。例如, 图 10-4 中, COMPU 和 DPMAG 都满足条件。

A	Y	G	S	U
D	O	M	R	A
C	P	F	A	S
X	B	O	D	G
W	D	Y	P	K
P	R	X	W	O

C	B	O	P	T
D	O	S	B	G
G	T	R	A	R
A	P	M	M	S
W	S	X	N	U
E	F	G	H	I

图 10-4 满足条件的密码

字典序最小的5个满足条件的密码分别是：ABGAG、ABGAS、ABGAU、ABGPG和ABGPS。给定 k ($1 \leq k \leq 7777$)，你的任务是找出字典序第 k 小的密码。如果不存在，输出 NO。

【分析】

本题是一个经典的解码问题。首先把不可能出现在答案中的字母排除。例如在上面的例子中，第1个字母只能是{A,C,D,W}，第2个字母只能是{B,O,P}，第3个字母只能是{G,M,O,X}，第4个字母只能是{A,P}，第5个字母只能是{G,S,U}。

不管第1个字母是多少，后4个字母都有 $3 \times 4 \times 2 \times 3 = 72$ 种可能，因此当 $k \leq 72$ 时，第1个字母是A，当 $72 < k \leq 144$ 时第1个字母是C，如此等等。再用同样的方法确定第2，3，4，5个字母即可。

由于 $k \leq 7777$ ，本题还有一个取巧的方法：直接按照字典序从小到大的顺序递归一个一个的枚举。虽然代码比递推法要长，但是由于思维难度小，往往能在更短的时间内写完、写对。

10.2.4 离散概率初步

关于概率有一套很深的理论，不过很多和概率相关的问题并不需要特别的知识，熟悉排列组合就够了。

第1个例子是：连续抛3次硬币，恰好有两次正面的概率是多少？用H和T来表示正面和背面（取自英文单词 head 和 tail），则一共有8种可能的情况：HHH、HHT、HTH、HTT、THH、THT、TTH、TTT。根据我们对硬币的认识，这8种情况出现的可能性相同，概率各为 $1/8$ 。用概率论的专业术语说，这里的{HHH、HHT、HTH、HTT、THH、THT、TTH、TTT}称为样本空间（Sample Space）。所求的是“恰好有两次正面”这个事件（Event）的概率。借助于集合的记号，这个事件可以表示为{HHT, HTH, THH}，其概率为 $3/8$ 。

提示 10-5：如果样本空间由有限个等概率的简单事件组成，事件 E 的概率可以用组合计数的方法得到： $P(E) = \frac{|E|}{|S|}$ 。

第2个例子是：如果一间屋子里有23个人，那么“至少有两个人的生日相同”的概率超过50%。为了简单起见，假定已知每个人的生日都不是2月29日。

尽管看上去复杂了许多，其实这个例子和抛硬币是类似的。每个人的生日是365天中

等概率随机选择的, 因此样本空间大小 $|S| = 365^{23}$ 。接下来需要计算“至少有两个人生日相同”的情况有多少种。这个数目不太好直接统计, 所以统计“任何两个人的生日都不相同”的数目, 然后用总数减去它即可。公式不难得到:

$$P(E) = \frac{|E|}{|S|} = \frac{|S| - |\bar{E}|}{|S|} = 1 - \frac{P_{365}^{23}}{365^{23}}$$

不管是 P_{365}^{23} 还是 365^{23} 都无法储存在 int 或者 long long 中, 但概率是实数, 并且此处并不需要太高的精度, 所以可以直接计算, 例如:

```
double P(int n, int m) {
    double ans = 1.0;
    for(int i = 0; i < m; i++) ans *= (n-i);
    return ans;
}
```

```
double birthday(int n, int m) {
    double ans = P(n, m);
    for(int i = 0; i < m; i++) ans /= n;
    return 1 - ans;
}
```

函数 birthday(365,23)的返回值为 0.5073, 即 50.73%。别高兴得太早, 我们来算一算 birthday(365,365)。直观上, 365 个人中几乎肯定会有两个人的生日相同, 因此 birthday(365,365)应该返回一个很接近 1 的值。可结果呢? 很不幸, 返回值为 -1.#INF0000——连 double 都溢出了。

解决方案是边乘边除, 而不是连着乘 m 次, 然后再连着除 m 次。例如:

```
double birthday(int n, int m) {
    double ans = 1.0;
    for(int i = 0; i < m; i++) ans *= (double)(n-i) / n;
    return 1 - ans;
}
```

本例说明: 正如数论和组合计数中要注意 int 和 long long 溢出一样, 在概率计算中要注意 double 溢出。顺便说一句, 这个“改进版”程序其实有个直接的概率意义:

$$P(E) = 1 - P(\bar{E}) = 1 - P(E_1)P(E_2)P(E_3) \cdots P(E_m) = 1 - \frac{n}{n} \times \frac{n-1}{n} \times \frac{n-2}{n} \cdots \frac{n-(m-1)}{n}$$

其中, E_i 表示“第 i 个人的生日不和前面的人重复”这个事件。上面的公式用到了这样一个结论: 如果有 n 个相互独立的事件, 则它们同时发生的概率是每个事件单独发生的概率的乘积, 像计数中的乘法原理一样。看上去很直观吧? 但严格的定义需要用到“条件概率”的知识。

条件概率。在概率计算中, 条件概率扮演了重要的作用。公式如下:

$$P(A|B) = P(AB) / P(B)$$

这里， $P(A|B)$ 是指，在事件 B 发生的前提下，事件 A 发生的概率，而 $P(AB)$ 是指两个事件 A 和 B 同时发生的概率。前面所说的两个事件 AB 独立就是指 $P(AB)=P(A)P(B)$ 。

条件概率中还有一个重要的公式，即贝叶斯公式： $P(A|B)=P(B|A) * P(A)/P(B)$

全概率公式。计算概率的一种常用方法是：样本空间 S 分成若干个不相交的部分 $B_1, B_2, \dots, B_n$ ，则 $P(A)=P(A|B_1)*P(B_1)+P(A|B_2)*P(B_2)+\dots+P(A|B_n)*P(B_n)$ 。

公式看上去复杂，但其实思路很简单。例如，参加比赛，得一等奖、二等奖、三等奖和优胜奖的概率分别为 0.1、0.2、0.3 和 0.4，这 4 种情况下，你会被妈妈表扬的概率分别为 1.0、0.8、0.5、0.1，则你被妈妈表扬的总概率为 $0.1*1.0+0.2*0.8+0.3*0.5+0.4*0.1=0.45$ 。使用全概率公式的关键是“划分样本空间”，只有把所有可能情况不重复、不遗漏地进行分类，并算出每个分类下事件发生的概率，才能得出该事件发生的总概率。

例题 10-9 决斗 (Headshot, ACM/ICPC NEERC 2009, UVa1636)

首先在手枪里随机装一些子弹，然后抠了一枪，发现没有子弹。你希望下一枪也没有子弹，是应该直接再抠一枪（输出 SHOOT）呢，还是随机转一下再抠（输出 ROTATE）？如果两种策略下没有子弹的概率相等，输出 EQUAL。

手枪里的子弹可以看成是一个环形序列，开枪一次以后对准下一个位置。例如，子弹序列为 0011 时，第一次开枪前一定在位置 1 或 2（因为第一枪没有子弹），因此开枪之后位于位置 2 或 3。如果此时开枪，有一半的概率没有子弹。序列长度为 2~100。

【分析】

直接抠一枪没子弹的概率是一个条件概率，等于子串 00 的个数除以 00 和 01 总数（也就是 0 的个数）。转一下再抠没子弹的概率等于 0 的比率。

设子串 00 的个数为 a ，0 的个数为 b ，则两个概率分别是 a/b 和 b/n 。问题就是比较 an 和 b^2 。前者大就是 SHOOT，后者大就是 ROTATE。

例题 10-10 奶牛和轿车 (Cows and Cars, UVa10491)

有这么一个电视节目：你的面前有 3 个门，其中两扇门里是奶牛，另外一扇门里则藏着奖品——一辆豪华小轿车。在你选择一扇门之后，门并不会立即打开。这时，主持人会给你个提示，具体方法是打开其中一扇有奶牛的门（不会打开你已经选择的那个门，即使里面是牛）。接下来你有两种可能的决策：保持先前的选择，或者换成另外一扇未开的门。当然，你最终选择打开的那扇门后面的东西就归你了。

在这个例子里面，你能得到轿车的概率是 $2/3$ （难以置信吧！），方法是总是改变自己的选择。 $2/3$ 这个数是这样得到的：如果选择了两个牛之一，你肯定能换到车前面的门，因为主持人已经让你看了另外一个牛；而如果你开始选择的的就是车，就会换成剩下的牛并且输掉奖品。由于你的最初选择是任意的，因此选错的概率是 $2/3$ 。也正是这 $2/3$ 的情况让你能换到那辆车（另外 $1/3$ 的情况你会从车切换到牛）。

现在把问题推广一下，假设有 a 头牛， b 辆车（门的总数为 $a+b$ ），在最终选择前主持人会替你打开 c 个有牛的门（ $1 \leq a \leq 10000$ ， $1 \leq b \leq 10000$ ， $0 \leq c < a$ ），输出“总是换门”的策略下，赢得车的概率。

【分析】

使用全概率公式。打开 c 个牛门后，还剩 $a-c$ 头牛，未开的门总数是 $a+b-c$ ，其中有